mcx

A Typst package for typesetting randomized multiple-choice exams.

Author: 1zumiSagiri
Version: 0.1.0

mcx

1. Introduction

The `mcx` package is a Typst implementation inspired by the LaTeX package mcexam. It allows educators to create randomized, multiple-version exams with automated answer keys, solution guides, and diagnostic tables. It also provides features for generating `Python` scripts to automate the exam generation process. `mcx` is inspired by LaTeX package `mcexam`, but is designed to be more flexible when user wants to insert code snippets or other content, leveraging Typst's powerful typesetting capabilities.

1.1. Features

- **Randomized Exams:** Create multiple versions of an exam with randomized question order and/or answer choices using a single `seed` value.
- **Flexible Shuffling:** Randomize question blocks (respecting connectivity logic) and answer choices.
- **Multiple Output Modes:** Seamlessly switch between student exams, answer keys, solution manuals, and exam generation concept overviews.
- **Automated Tables:** Generate question and answer permutation tables to track how content moved between versions.
- **PDF Splitting:** Includes a helper script to automatically split a single compiled PDF into individual version files.

2. Getting Started

To get started with `mcx`, you can import the package into your Typst document:

```
#import "@preview/mcx:0.1.0": *
```

3. Defining Question List

Use the provided macros to define your question list using the `mc-question` and `mc-answer` constructors:

```
#let my-questions = (
  mc-question(
    [...],
    (mc-answer(...), ...)
  ),
  ...
)
```

To build the exam using the defined question list, use the `mc-questions` function:

```
#mc-questions(my-questions, output: "exam", number-of-versions: 4, version: 1, seed:
40)
```

The `mc-questions` function takes the question list and generates the specified output based on the provided parameters:

- `output`: Specify the type of output to generate:
  - `"exam"`: Generates the student version of the exam with questions and answer choices.
  - `"answers"`: Generates `exam` with the correct answers.
  - `"key"`: Generates the answer key table for the versions of the exam.
  - `"concept"`: Generates a concept overview that explains the underlying logic when shuffling questions and answers, which can be useful for educators to understand the randomization process.

- `number-of-versions`: The number of different versions of the exam to generate. It will generate four versions by default.
- `version`: Specify a particular version to generate, and add the version number to the top of the file. It will generate the first version by default.
- `seed`: A number used to initialize the randomization process. Using the same seed will produce the same randomization, allowing for reproducibility. If not specified, it will default to `1`. The randomization process is based on Typst package `suiji`.
- `randomize-questions`: A boolean parameter that controls whether the order of questions should be randomized. If set to `true`, the questions will be shuffled based on the specified `seed`. If set to `false`, the questions will appear in the order they are defined in the source code across all versions. By default, this parameter is set to `true`.
  - The questions grouped together by the `follow` parameter in `mc-question` will be treated as a single block when `randomize-questions` is enabled, meaning that the entire block will be shuffled together while maintaining the internal order of questions within the block. This allows for maintaining logical connections between questions while still randomizing their order in the exam Section 4.
- `randomize-answers`: Same as `randomize-questions` but for answer choices.
- `config`: Provide a custom configuration for the output mode.

4. Defining Question `mc-question`

```
#let qs = (
  mc-question(
    [
      What is $2+2$?
    ],
    (
      mc-answer([$4$], mark: "correct"),
      ...
    ),
    permute: (type: "fixlastn", n: 2),
    instruction: [Some easy math question.],
    explanation: [Self-explanatory.],
  ),
  mc-question(
    [
      Given function: $f(x) = x^3 - 2x^2 + 5x - 7$, calculate $f'(2)$.
    ],
    (
      mc-answer([$9$], mark: "correct"),
      ...
    ),
    permute: "permuteall",
    follow: true,
    explanation: [Some explanation for this question.],
    notes: [This question is connected to the previous one, so it will be shuffled
together with it when `randomize-questions` is enabled.],
  )
)
```

`mc-question` is the primary constructor for defining a multiple-choice question. It takes `body` which is a content block for the question text and an array of `mc-answer` for the answer choices. There are also optional parameters for controlling the permutation of questions and answers, as well as providing explanations and notes:

- `permute`: Package will randomize the order of answers based on the specified permutation type. Details on the available permutation options for answers are provided in Section 4.1.
- `follow`: The questions will be grouped together with the next `follow` questions, and the entire group will be permuted together based on the specified `permute` option. This allows for maintaining logical connections between questions while still randomizing their order in the exam.
- `instruction`: Provide an `instruction` that will be displayed at the beginning of the group of questions. This is useful for providing context or specific instructions related to the grouped questions.
- `explanation` and `notes`: These parameters allow you to provide additional content for each question that can be included in the solution manual or other output modes. The `explanation` is intended for detailed solutions, while `notes` can be used for any additional information or hints related to the question.

4.1. Permuting Answers

The `permute` parameter in `mc-question` allows you to specify how the answer choices should be randomized. The available options for `permute` are:

- `"permuteall"`: Package will use the provided `seed` to randomize the order of answers for each version, resulting in different answer orders across versions.
- `"fixlast"`: Similar to the `"permuteall"` option but keeping the last answer fixed in its position across versions.
- `(type: "fixlastn", n)`: Similar to the `"fixlast"` option but keeping the last `n` answers fixed in their positions across versions. Will clamp `n` to `[1, number of answers]`.
- `"ordinal"`: The order of answers will either be the same as the order they are defined in the source code or reversed.
- `"permutenone"`: The package will not randomize the order of answers, and they will appear in the same order across all versions.
- User can also specify a custom permutation function by passing an arrary of numbers representing the new order of answers. It can also be an array of arrays of numbers to specify different custom permutations for each version.

5. Defining Answer `mc-answer`

When defining answers for `mc-question`, `mc-answer` should be provided for each answer choice. The whole block of answers will be in an array as the parameter of `mc-question`. The `mc-answer` constructor takes the following parameters:

- `body`: The content block for the answer choice text.
- `mark`: An optional parameter to indicate if the answer is correct.
  - If set to `"correct"`, the package will recognize this answer as the correct choice when generating answer keys and solution manuals. If not specified, the answer will be treated as incorrect by default.
  - It can also be set to a number to indicate the point value of the answer, which will be reflected in the solution manual and answer key outputs.

6. Data Flow Process

In case you are interested in understanding the internal workings of the `mcx` package, here is a high-level overview of the data flow process that occurs when you run the exam generation script:

- **Data Collection:** All objects created via `mc-question` are gathered into a single array.
- **First Layer Shuffling (Question Order):**

    - The system identifies questions marked with `follow: true` and bundles them with the preceding question into indivisible "blocks".
    - Subsequently, the order of these blocks is randomized based on the defined `seed` and the specific `version` number.
- **Second Layer Shuffling (Answer Order):**
    - The program iterates through each individual question and shuffles its internal `answers` array according to its specific `permute` attribute (e.g., `permuteall`, `fixlast`).
- **Mapping and Recording:**
    - The system automatically generates a mapping table to track the relationship between original and displayed positions (e.g., Question 1 in Version A corresponds to Question 5 in the source data; its Option A corresponds to original Option C).
    - This map is ultimately used to generate the "Question Permutation Table" and the "Answer Key".
- **Rendering and Output:**
    - Based on the selected `output` mode (such as `exam`, `answers`, or `concept`), the system determines whether to render correct answer markers, solution explanations, or point values.